FPGA Chess: A Hardware Implementation and Recreation of Chess for FPGA Platforms

Umair Irshad
College of Engineering
University of Georgia
Athens, GA
ui94235@uga.edu

Nathan Park
College of Engineering
University of Georgia
Athens, GA
ngp29874@uga.edu

***Abstract* — The project presents a hardware-based recreation of the game of chess, implemented on the Nexys A7-100T FPGA board as FPGAs can manage complex, rule-based systems like chess in real time. Players interact using a USB mouse, and the game board and pieces are rendered via VGA output. The implementation supports standard rules, including legal movement, turn management, and detection of check and checkmate. This project explores the capabilities of digital design for interactive applications, highlighting the potential of FPGAs for low-latency, self-contained game systems.**

I. INTRODUCTION

A. Brief Explanation of the Game of Chess

Chess is a strategy board game in which two players take turns in moving one of their 16 pieces with the goal of capturing the opponent's king. The board consists of an 8x8 grid of alternating light and dark squares. Each player starts with a set amount of pieces of various types — eight pawns, two rooks, two knights, two bishops, one queen, and one king. Each piece moves differently depending on its type and only one piece is permitted at one square. The pawn typically can only move one square forward and cannot take with this move, unlike most other moves in the game. The pawn can also move diagonally forward-left and/or diagonally forward-right if and only if an opposing piece is present there. Upon its first move, the pawn can also move two squares forward. The rook can move any number of squares up, down, left, or right and is able to take opposing pieces. However, the rook cannot pass any pieces to move to the desired square. The knight can move in an "L" shape — two squares in any direction and one square perpendicular. The knight's move has the unique quality of being able to "jump" over other pieces. This move can also take other pieces. The bishop moves similarly to the rook, but diagonally rather than straight. The queen can move identically to the rook and the bishop. Finally, the king can move one square in any direction. Players aim to move pieces to target the opponent's king, and the event in which a piece is moved that directly poses the threat of capture to the king is called check. When the king is under check and cannot escape capture, the game is ended via checkmate. Players cannot expose their own king, or make moves that do not evade capture while under check.

B. Overview of the Nexys A7 100T Platform

The Nexys A7 100T was the board used for this project. It is a versatile FPGA development board featuring Xilinx's Artix-7 FPGA, onboard memory, USB/Ethernet interfaces, and a variety of built-in peripherals. It supports designs from basic logic circuits to advanced embedded systems without requiring additional components. [1] The I/O utilized for this project is the VGA output, one of the PMOD outputs, and the four button inputs.

C. Design Objectives and Use Cases

The minimum project requirements are as follows: (1) The user can control the cursor using the on-board buttons. (2) Pieces can be selected and moved. (3) The pieces move accurately according to the rules of chess. (4) The connected speaker plays a sound effect

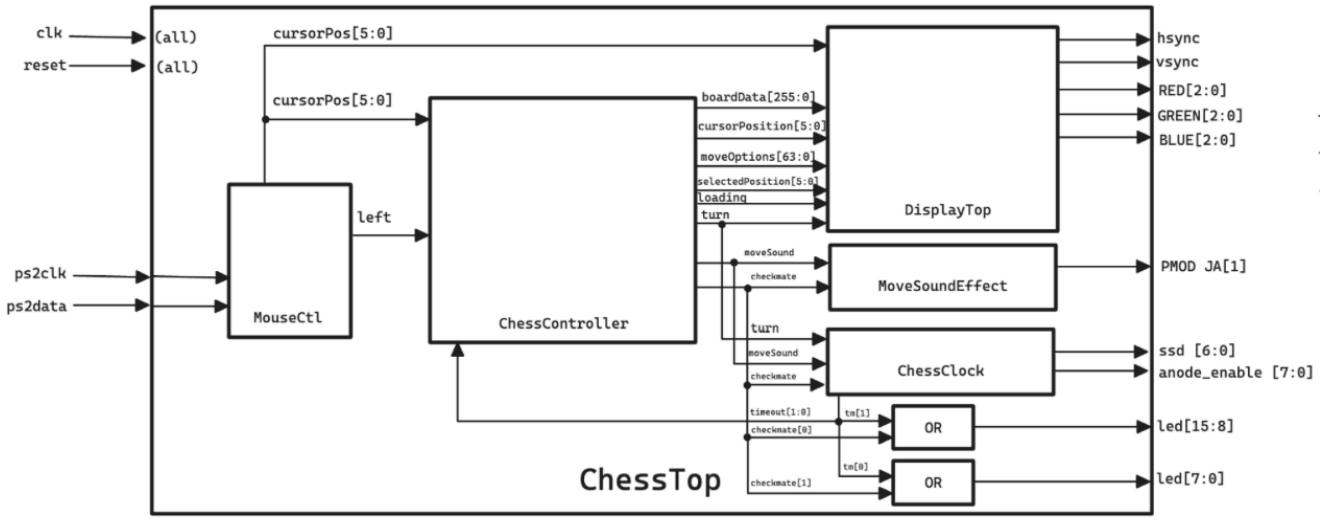


Figure 2.1: *Top Module Block Diagram*

indicating that a move has been made. (5) The connected VGA display shows a chess board, chess pieces, the cursor, and the move options. These goals were met.

The requirements for a fully functioning project are as follows: (1) All previous minimum requirements. (2) The system recognizes check and checkmate. (3) Incorporating memory into either/both the game logic and/or display controller. (4) The game is replayable through a reset button. THese goals were met.

The stretch goals for the project are as follows: (1) Game clock using the seven segment displays. (2) A mouse can be used through the USB port as a replacement for the buttons. All stretch goals were accomplished.

II. System Design

A. Block Diagram

The overall design of the project is visualized in Figure 2.1.0. The main top module instantiates five main modules, which are the mouse control, chess controller, display top, move sound, and chess clock modules. The mouse control module utilizes digilent's ps2 interface to translate inputs from a USB mouse to a cursor position on the board and left click signal. The chess controller module will receive these inputs and output the necessary data to be read by the display top module. The display top module handles converting the game data into the VGA display outputs hsync, vsync, and RGB. The display top module draws the board, pieces, move options, and the cursor. The chess clock module keeps track of each player's remaining time and outputs to the seven segment displays. Additionally, upon timeout or checkmate, the corresponding LEDs will toggle for the winning player. Finally, the move sound effect module handles the output of a sound cue when the appropriate signal is received from the chess controller.

Figure 2.1.1 visualizes the overall design of the chess controller. The following modules are instantiated by ChessController: ChessBoard, ChessPiece, InstructionHandler, CheckAllController, CheckmateController, and ModifyOptionsHandler. Each module serves a different function that allows the chess controller top module to compute game logic.

B. Chess Logic

The chess controller module is designed with the following features: (1) Initializes starting positions. All pieces can move as intended. (2) Can use button inputs to move pieces. (3) Recognizes check and checkmate using simulations. (4) Indicates when calculating using the loading output. (5) The first/last 8 LEDs are used to indicate the winning player. These features are achieved through the other modules within the chess controller.

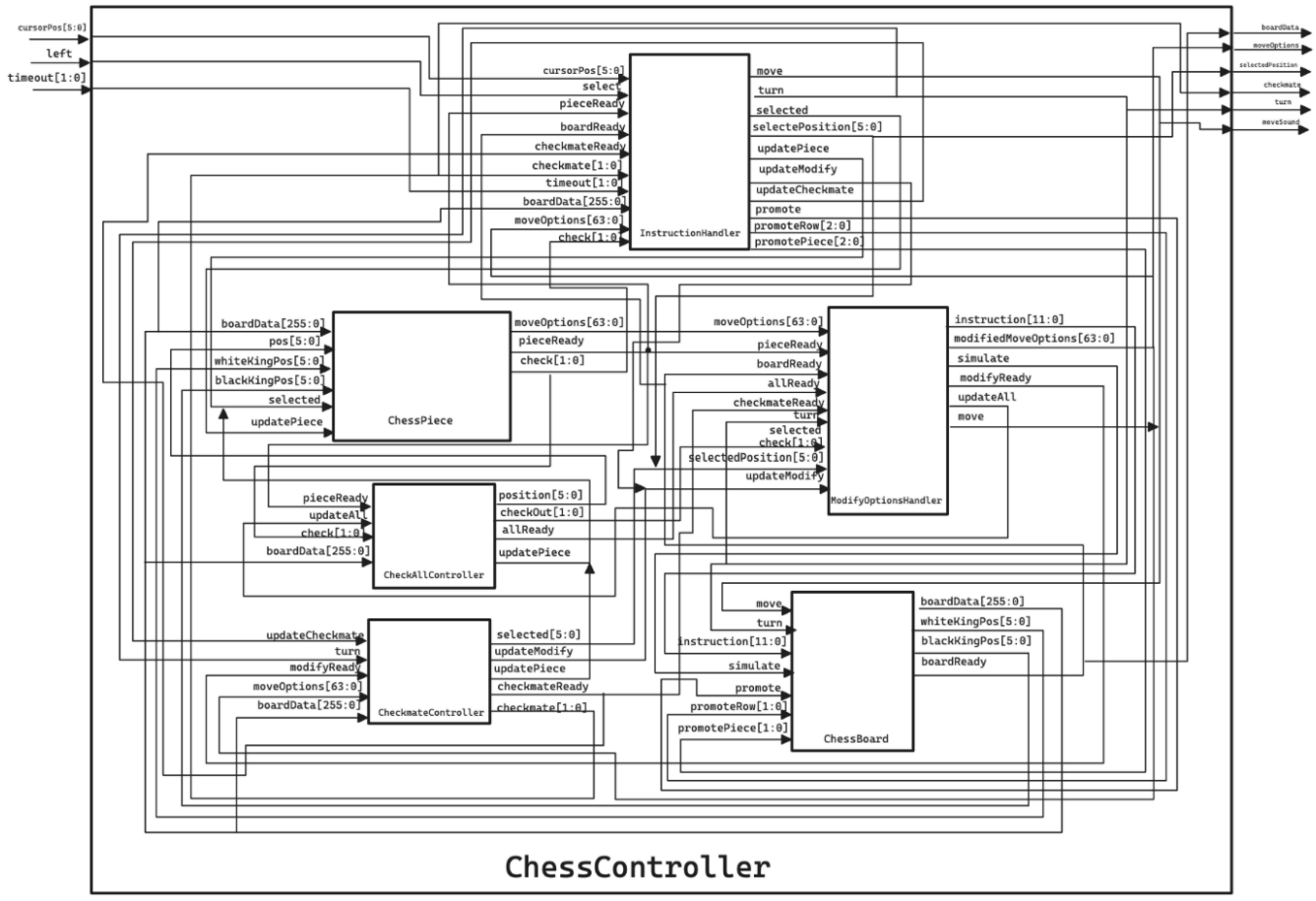


Figure 2.1.1: *Chess Controller Block Diagram*

The chess piece module handles calculating the options that a given piece can move through a position input. It is designed with the following features: (1) Calculates and outputs the current move options. (2) Reusable for checking the move options for every piece on the board. (3) Signals when ready. This module was originally used in 32 instantiations, but this design proved to be too resource intensive. Alternatively, the module was redesigned to be reusable for all pieces sequentially. This new design is much less resource intensive, yet requires a new module to determine check — CheckAllController.

The chess board module is designed with the following features: (1) Initializes, manages and outputs the board data. (2) Places starting pieces upon initialization. (3) Handles parsing and carrying out instructions. Instructions are already validated, but some additional checking is done here. (4) Sends an update pulse after carrying out an instruction. This pulse is received by the pieces to recalculate moves. (5) The board can save and restore the board data. This will be used during the simulations for check and checkmate controllers. The board data has a size of 256 bits — reduced from 512 from previous iterations — and holds the piece data for each square on the board.

The instruction handler module takes user input of the cursor position and select, and it outputs an instruction to send to the board. It is designed with the following features: (1) Handle setting the starting square after the user selects a piece. (2) Check the board data and cursor position upon pressing the select button (left mouse button). (3) Generates move options by updating the chess piece module with the starting position. (4) Handles checking if the target square is a valid move option. (5) Send the resulting instruction to the chess board module to make changes to the board.

The check all controller module iterates through each position on the board, and quickly calculates the move options by sending the position and update signals to the chess piece module. Once ready, the piece will output whether the current position's piece is checking the opponent. In other words, this module determines if there is a check currently on the board for either the white pieces or the black pieces.

However, not all move options are legal when considering check, and the modify move options handler module addresses this issue. It works as follows: When a piece is selected, simulations are run for each option available to the piece. This is done by setting the 'simulate' signal to HIGH for the chess board module, saving the current board data. Then, the required instruction for making the move is sent. If the simulated instruction results in a defending check, the option is removed. The check is determined by updating the check all controller module. After a simulation, the 'simulate' signal is set back to LOW, restoring the board data to its actual state. After all simulations are completed, the move options have been modified to only include moves that do not result in a defending check. This accounts for situations under check and illegal moves.

In order to account for checkmate, the checkmate controller module simulates a selection of every defending piece when under check until a possible move is found or all pieces are simulated. This process only starts when a check is made. Then, the checkmate controller module sends a simulated selection of every piece on the board sequentially. The previous modules take care of removing move options that result in a check, so, if no move option is found for any defending piece, then it is checkmate.

C. Chess Clock

The chess clock runs outside of the chess controller, and uses the move, checkmate, and turn outputs of the chess controller. The clock only starts running once the first move is made, and decrements the timer that the current turn represents. The chess clock is run on a 100MHz clock, so a counter increments up to one hundred million before decrementing either clock. If at any point a timer reaches zero or a checkmate occurs, then the clock ends operation. The white and black clocks are displayed on the left four and right four seven segment displays, refreshed at 1000 Hz.

D. VGA Output (Display)

To display the game, the VGA output that is available on the board was used. A custom VGA controller module was built that handled all timing, resolution, and synchronization signal generation (h-sync and v-sync). It also outputs pixel coordinates (x, y) and a valid signal used by all other display modules.

The rendering logic was split into multiple modules instantiated within the display top module. Each module determines whether the xy cursor is in an active region of its drawing, and these active outputs can be used to layer the drawings appropriately.

Plainly, the rendering logic works as follows: At a certain coordinate, if the cursor is active then draw the color of the cursor here. If the cursor is not active, then if the piece is active, draw the color of the piece here. If the piece is not active, then if the board is active, draw the color of the board here. If the board is not active, draw black. This sequence drives the entirety of the VGA rendering logic, and each module only needs to decide whether the current coordinate is in an active region to draw, and the color that must be drawn there.

The chessboard VGA module handles drawing an 8x8 chessboard grid with alternating tile colors for dark and light squares. At a set tile size, the current color of the board can be determined through dividing the current coordinates by the tile width and height.

The chessPieces module is responsible for rendering each piece as a sprite using block RAM. The pieces are drawn with color and a black outline, depending on their type, color, and position. This module also flips the board orientation based on the current player's turn.

The cursor VGA module renders a rectangular highlighted box that is displayed at the location calculated through the mouse control module and is always drawn on top of the board and pieces.

The move options VGA module displays a yellow highlight or pathway over tiles representing valid move options for the selected piece to aid user interaction and decision-making.

E. Mouse

Initially, a custom mouse implementation was created by writing a custom PS/2 interface module, similarly to how a keyboard is handled with the PS/2 interface. The PS/2 interface allowed communication with the mouse so that packets of data could be received reliably and used to decode movement and button press events. However, the primary challenge encountered was debouncing the mouse input. Unlike a keyboard where key presses are discrete and relatively slow, mouse movement generates a rapid stream of packets that must

be interpreted smoothly. Our custom module struggled to eliminate jitter and bouncing, which made the cursor motion appear erratic and unstable.

To address the debouncing, the official Digilent demo program for the Nexys A7 board was used as a reference for handling mouse integration, which proved to be extremely complex and refined. In order to improve the overall user experience, Digilent's VHDL mouse control module was incorporated into the design, efficiently combining VHDL and verilog.

This hybrid approach worked seamlessly, allowing us to achieve stable and responsive mouse tracking, identical to the behavior in the Digilent demo.

In the design, the mouse's left-click signal is used to trigger piece selection and movement, and the X/Y position data to control cursor positioning on the board. This ensured a smooth user experience while saving significant development time by leveraging existing, well-tested IP.

F. Speaker

In order to incorporate sound effects into the design, the PMOD JA port was used to play a quick sound every time a player moves a piece. The move sound effect module generates a 400 Hz square wave, upon detection of a HIGH for the move sound signal (which happens every time a move is made). This square wave lasts for exactly 0. 2 seconds.

The way it works is pretty simple: when moveSound pulses, the module starts counting clock cycles and flips the speaker pin on and off fast enough to create the square wave sound. After about 20 million cycles (around 0.2 seconds at 100 MHz), the sound automatically stops. This gives a quick "blip" noise that makes the game feel a lot more responsive without being annoying.

The output was directly assigned to JA[1] on the Nexys A7 board, and added a constraint in the XDC file to map it correctly. It ended up working really smoothly and added a nice finishing touch to the user experience.

III. Implementation

A. Development Environment

We developed this project using the Xilinx Vivado Design Suite, which allowed HDL-based development and simulation targeting the Nexys A7-100T board. We also used Icarus Verilog (iVerilog) and GTKWave to perform all the game logic testing and debugging before bringing the design into Vivado to test the VGA output. We used the Verilog hardware description language for all the modules. Vivado's integrated simulation environment also supported additional testing and analysis.

B. Integration of Modules

Although each chess controller module works in simulation, there are many issues that are both caught and not caught by Vivado that must be addressed for a proper bitstream generation. As mentioned previously, the original integration of the chess piece module was to instantiate it 32 times — one for each piece. This proved to use over two-hundred-thousand LUTs, which is much more than the hardware available. Additionally, many timing issues may occur when reading and writing to registers on the same clock cycle, possibly by different modules. This is caught by Vivado when these registers are passed as wires to other modules — a combinatorial loop error (DRC LUTLP-1). This error is only thrown during bitstream generation, but can be fixed by intentionally passing intermediate values through new registers, which the output can be set to on its own clock cycle. However, this error is not caught by Vivado if there is no wired connection between modules that is affected, and results in issues with state machine logic that otherwise works in simulation. Later, this issue was also addressed in the VGA implementation to prevent screen tearing.

C. Hardware Mapping and Constraints

The hardware design follows the Nexys A7 XDC constraints, we mapped the directional and center buttons to control the cursor and select pieces. VGA output pins were used to display the board and chess pieces on the monitor and we also utilized the PMOD ports to connect a speaker that would handle sound effects through PWM audio. A clocking wizard was

instantiated to provide a 109 MHz clock for VGA timing and a 100 MHz system clock for game logic and synchronization.

D. VGA implementation

The VGA controller is driven by a 108 MHz pixel clock, generated using the Clocking Wizard from Vivado's IP catalog, to match the 1280x1024 VGA resolution.

All VGA modules are synchronized using the valid signal from the VGA controller and the pixel clock to ensure no tearing or flickering occurs. Each module contributes its own pixel color signals, which are multiplexed to produce the final red, green, and blue VGA outputs. To ensure a smooth, flicker-free display, we updated the red, green, and blue signals in an always block driven by the pixel clock.

Regarding the loading output of the chess controller, the desired functionality would be to block changes in the display when the chess controller is loading (simulating moves and board positions). However, in implementation, the chess logic computes quickly enough such that the loading output does not need to be handled,, and no simulations are visible at all throughout actual gameplay.

D. Debugging and Testing Process

Debugging was done both in simulation and hardware. Modules like ChessBoard, CursorController, and CheckController were first tested in a simulation to verify FSM behavior and move logic. For completed chess logic, the chess controller testbench allowed simulated games to extensively test various move options and positions. On the board itself we used on-board LEDs to help trace module activity during gameplay. Regarding the testing of the display modules, predetermined registers for board data and chess logic outputs were used to ensure that the board, pieces, and move options were displaying correctly. These debugging and testing strategies allowed the team to save time and resources, rather than generating a bitstream

IV. Evaluation & Results

A. Functional Testing

During final demonstrations, all desired features functioned as expected, including the mouse, VGA display, chess logic, seven segment display-driven clock, and sound effects. The following video demonstrates these features:

https://drive.google.com/file/d/1MmDmGrgtNYTDgenpOOMRkVBCJaKvtMi6/view?usp=drivesdk

B. Performance Metrics

The design uses a small portion of the Nexys A7 100t resources. According to our utilization report we only used about 17% of LUTs, 1% of flip flops and 6% of block RAM which means there is still plenty of room for any new features that we might wanna add in the future. We also used around 22% of the available IO mostly USB, VGA, dip switches and PMOD.

The MMCM which is the clocking resource is only using one instance to generate our VGA pixel clock which comes out to be about 17% of the available MMCM on the board

V. Conclusion

The project pushed us to our limits and allowed us to explore the capabilities of FPGA development. More than any other assignment, this project allowed the team to utilize and improve their creativity and technical skills. Unlike most FPGA course projects, it gave us hands-on experience working with VGA timing and memory mapped sprites.We feel the resulting product of our design met our requirements and accomplished the stretch goals we set for ourselves — including the incorporation of a USB mouse and real-time game clock. All in all, we found that the design process was rewarding and that playing our own version of this classic game was even more enjoyable.

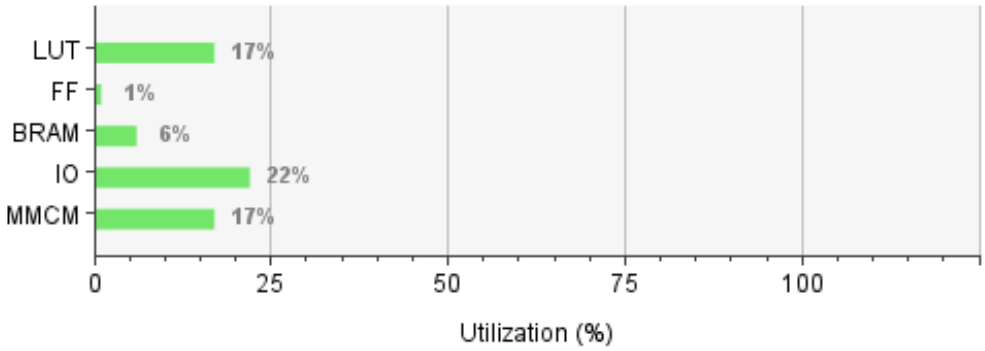
APPENDIX

https://github.com/Herring-UGACSEE-4280/group-8/tree/main/FinalProject

A1: *GitHub Link*

Resource	Utilization	Available	Utilization %
LUT	11020	63400	17.38
FF	1405	126800	1.11
BRAM	8	135	5.93
IO	47	210	22.38
MMCM	1	6	16.67

A2: *Resource Utilization Chart*



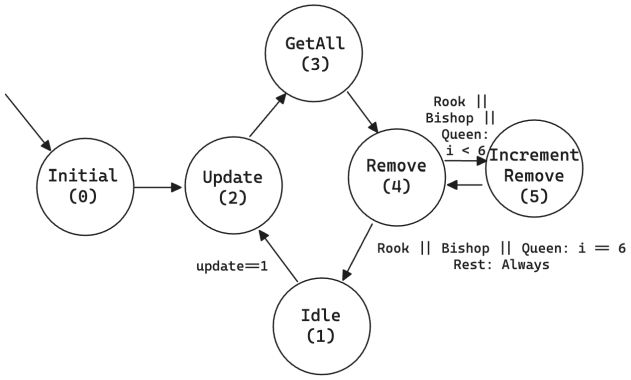
A3: *Resource Utilization Graph*

Design Timing Summary

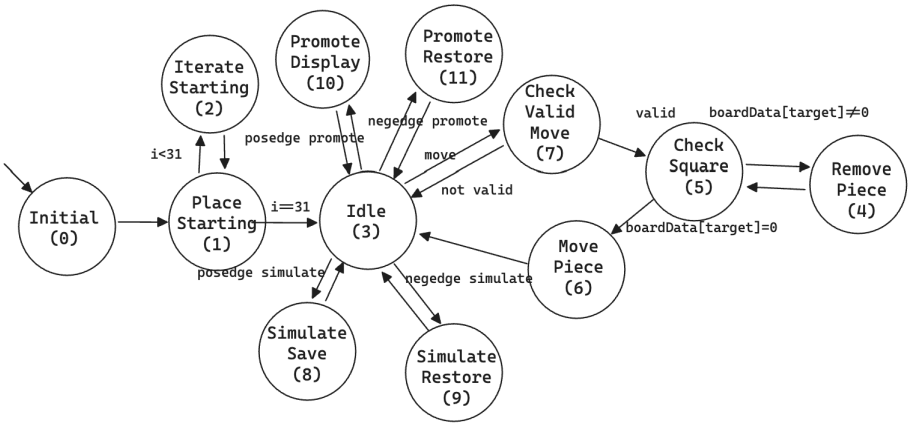
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	-11.701 ns	Worst Hold Slack (WHS):	0.137 ns	Worst Pulse Width Slack (WPWS):	3.000 ns
Total Negative Slack (TNS):	-716.195 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	137	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	2917	Total Number of Endpoints:	2917	Total Number of Endpoints:	1419

Timing constraints are not met.

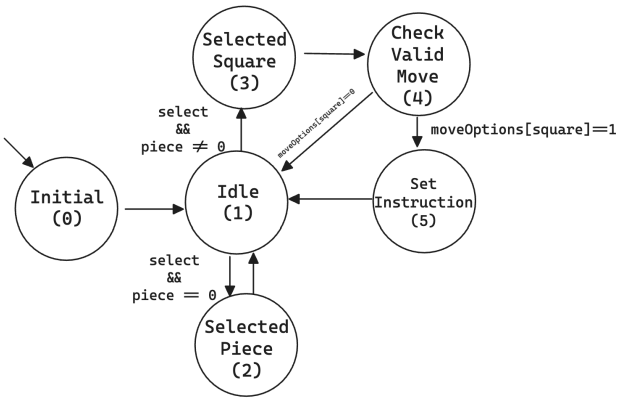
A4: *Design Timing Summary*



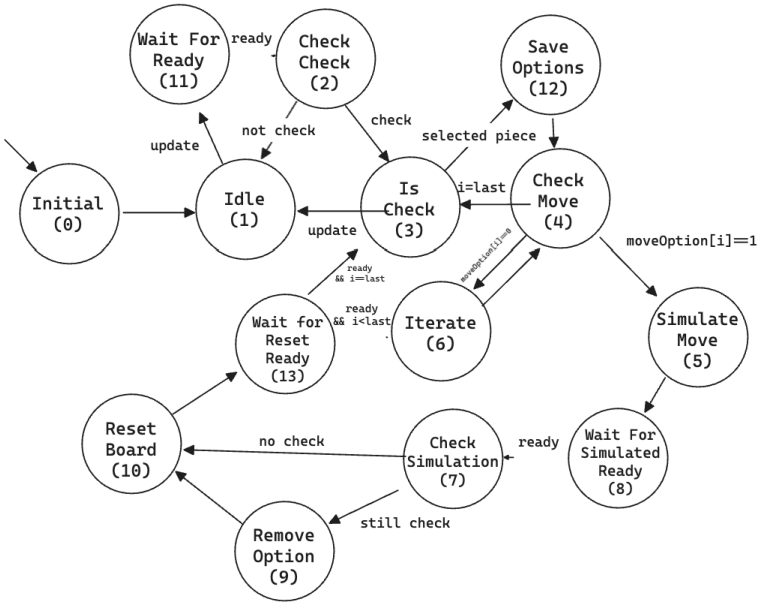
A5: *ChessPiece Module FSM*



A6: *Chess Board Module FSM*



A6: *Instruction Handler Module FSM*



A7: *Modify Options Handler Module FSM*

REFERENCES

[1]Digilent, “GitHub
- Digilent/Nexys-A7-100T-OOB,” GitHub, Nov. 2018. https://github.com/Digilent/Nexys-A7-100T-OOB (accessed Apr. 26, 2025).

[2] Digilent, “Nexys A7 Reference Manual,” *Digilent Documentation*, [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual. [Accessed: Apr. 7, 2025].